

AI Master Project

MailTaskAgent

메일을 업무로 해석하고 다음 행동을 결정하는 개인 업무관리 Agent

백준현 | 09285

AI Master 7기 | 멘토 이유경

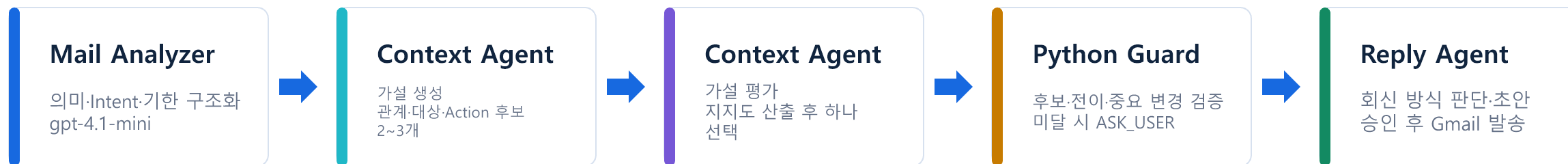
최종 발표

발표 10분 이내 | 시연 영상 별도 제출 5분 이내

프로젝트 개요

같은 업무 요청이 여러 메일과 Thread에 흩어져 도착합니다

무엇이 새 업무이고 무엇이 기존 업무의 변경인지 사람이 매번 판단해야 합니다. 놓치면 기한을 넘기고, 잘못 붙이면 엉뚱한 업무가 바뀝니다.



핵심 기술

회사 gpt-4.1-mini · Python + Pydantic · SQLite Task Context · Gmail API · Streamlit

검증 결과

28/28

Action 단계

3/3

Agent Context Live

20/20

Gmail 수용시험

보조 검증 | pytest 228 passed · 중복 재조회 35/35

Reply Planning 3/3 · Draft 1/1 · 사용자 승인 실제 Gmail 발송 1건

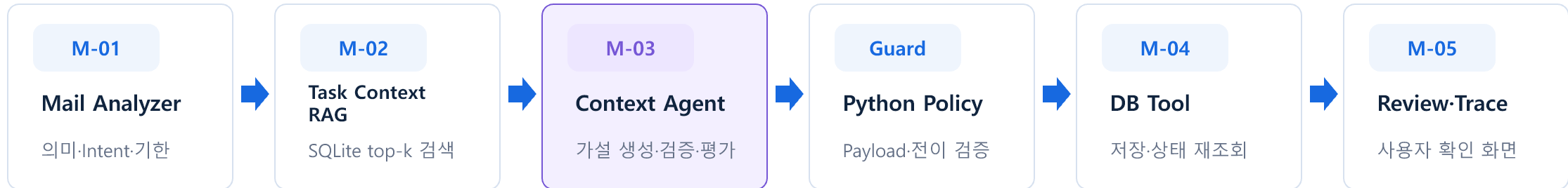
자동 반영 비율 | 정의된 합성 15 Case에서 자동 반영 8/15(53.3%) · 사용자 확인 7/15(46.7%)

위 수치는 정의된 Case 범위의 결과이며 실제 Mailbox 전체 성능이 아닙니다. 사용자 체감 시간은 미측정입니다.

핵심 메시지 | 규칙으로 열거할 수 없는 "이 메일이 어느 업무인가"만 Agent가 판단하고, 실행 권한은 Python과 사용자가 가집니다.

기술 아키텍처와 Agent 판단 구조

M-01~M-05 Workflow는 안전한 실행 뼈대, 규칙으로 확정할 수 없는 관계·대상·Action은 Agent가 판단



제한적 ReAct | Observe → Retrieve → Reason → Act → Observe (저신뢰·모호하면 Query Rewrite 후 재검색·재판단 최대 1회)

Task 관계 동일 Thread는 Metadata 규칙 다른 Thread·다른 표현은 후보 2~3개를 비교해 관계·대상·Action 선택

회신 실행 Reply Agent 판단 → 사용자 입력 → LLM 초안 → 승인 Gmail 발송 → WAITING_REPLY

기술 선택 이유

gpt-4.1-mini

선택 이유 | 메일 의미와 Task 관계는 규칙으로 열거할 수 없지만 출력은 고정 Schema여야 합니다

핵심 활용 | 관계·Action을 Pydantic 계약으로 받고, 계약 위반은 재시도 루프 안에서 재생성

SQLite Task Context RAG

선택 이유 | 찾을 대상이 외부 문서가 아니라 내 활성 Task·최근 Mail·History·과거 결정입니다

핵심 성과 | 제한 top-k 검색으로 Live 15 Case Action 28/28. Vector DB는 쓰지 않았습니다

Python + Pydantic Guard

선택 이유 | LLM이 DB를 직접 바꾸면 비용이 업무 데이터 오변경입니다. 상태가 닫혀 있어 LangGraph 없이 충분했습니다

구현 포인트 | 후보 ID·상태 전이·중요 변경을 검증하고 완료·취소·기한 단축은 항상 승인

M-01은 모든 Mail 의미를 LLM으로 구조화하고, 관계·Action 선택은 STRUCTURED_RAG 경로에만 적용합니다.

핵심 기술 과제와 검증

왜 고정 규칙과 단일 결론 Agent만으로 부족했는가

Agent가 결론 하나만 내면 후보를 건준 것인지 설명을 붙인 것인지 알 수 없습니다. 멘토 피드백도 "Rule Base처럼 보인다"였습니다. 왜 이 접근인가 — 틀렸을 때 비용이 업무 데이터 오변경이라, 넓게 펼치는 대신 후보를 제한하고 Guard와 사용자 승인으로 막았습니다.

실제 Agent 판단 흐름

적용 기법 | Bounded Multi-Hypothesis Deliberation + 제한적 ReAct Self-Correction

Deliberation은 한 번의 Reason 안에서 후보를 건주는 단계이고, ReAct는 그 판단을 Retrieve-Act-Observe로 감싸는 바깥 Loop입니다. Tree of Thoughts(Yao et al., NeurIPS 2023)의 후보 생성-평가 분리를 참고하되 후보 2~3개·평가 1회·재검색 1회로 제한했고, Full Tree of Thoughts 구현은 아닙니다.

1	Observe · Retrieve	현재 Mail 관찰 + SQLite Task Context RAG로 활성 Task·최근 Mail 3건·History 5건 검색
2	Reason · 가설 생성	가능한 관계·대상·Action 후보 2~3개를 근거와 함께 제시 (점수는 매기지 않음)
3	Reason · 검증·평가	Python이 후보 계약을 검증하고, 별도 평가 단계가 지지도를 매겨 하나를 선택
4	Self-Correction	모호·저신뢰이거나 선택 차이가 작으면 Query Rewrite → Context 재검색 → 재판단, 최대 1회
5	Act · Observe	Python Guard 검증 → DB 실행 또는 ASK_USER → 실제 저장 결과를 다시 조회해 확인

같은 Case·같은 코드에서 속고만 졌다 켄 비교

기존 15 Case

15/15 · Action 28/28

속고 On/Off 동일, 시간만 증가

속고 전용 4 Case

재현되는 차이 1건

DLB-02: On 3/3 통과, Off 3/3 실패

실제 Gmail 재현

새 메일 4건 처리

2건 Task 생성, 2건 ASK_USER

Next Step | Gmail 검증 결과를 바탕으로 Outlook-Graph Adapter와 사내 인증·운영 서버로 전환

실패 | 계약 위반 응답을 재시도 없이 실패 처리해 판단 4건 유실

개선 | 검증을 재시도 루프 안으로 이동, 유실 0건